

SQL Analytical Tasks

Step-by-Step Guide

Tank54 Group | Oracle HR Schema

What is in this book?

This guide has two parts. **Part 1** shows 5 examples. Each example has a task, step-by-step explanation, and the SQL query. **Part 2** has 15 practice tasks for you to solve.

Topics: SELECT, WHERE, JOINS, GROUP BY, HAVING, Aggregate Functions, Subqueries, ORDER BY

Oracle HR Schema - Tables

We use the Oracle HR database. This database has information about employees, departments, jobs, and salaries. Here are the main tables we use in this guide.

Table Name	What is inside?
EMPLOYEES	Employee ID, first name, last name, email, phone, hire date, job, salary, manager, department
DEPARTMENTS	Department ID, department name, location, manager
JOBS	Job ID, job title, min salary, max salary
JOB_HISTORY	Employee, start date, end date, job, department
LOCATIONS	Location ID, street address, city, country

Important Columns in EMPLOYEES table:

Column	Type	Example
EMPLOYEE_ID	Number	100
FIRST_NAME	Text	Steven
LAST_NAME	Text	King
HIRE_DATE	Date	17 - JUN - 03
JOB_ID	Text	AD_PRES
SALARY	Number	24000
DEPARTMENT_ID	Number	90
MANAGER_ID	Number	(null or number)

PART 1: Examples

In this part, we show 5 examples. Each example has 3 parts: the task, the steps, and the SQL query. Read each step carefully. Try to understand the logic before you look at the SQL code.

Example 1: Average Salary by Department

Task:

Find the average salary for each department. Show only departments where the average salary is more than 10000. Order the result by average salary from high to low.

Step-by-Step:

Step 1: What tables do we need? We need EMPLOYEES (for salary) and DEPARTMENTS (for department name).

Step 2: We need to connect the tables. EMPLOYEES.DEPARTMENT_ID = DEPARTMENTS.DEPARTMENT_ID. This is an INNER JOIN.

Step 3: We need the average salary for each department. So we use GROUP BY department name. And we use AVG(salary).

Step 4: We only want departments with average salary > 10000. After GROUP BY, we use HAVING (not WHERE).

Step 5: Order by average salary from high to low: ORDER BY ... DESC.

SQL Query:

```
SELECT d.department_name,
       ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
INNER JOIN departments d
      ON e.department_id = d.department_id
GROUP BY d.department_name
HAVING AVG(e.salary) > 10000
ORDER BY avg_salary DESC;
```

Example 2: Employees Who Earn More Than Their Manager

Task:

Find all employees whose salary is higher than their manager's salary. Show employee name, employee salary, manager name, and manager salary.

Step-by-Step:

Step 1: We need to find each employee and their manager. The MANAGER_ID in EMPLOYEES points to another EMPLOYEE_ID. So we need to join EMPLOYEES with itself. This is called a SELF JOIN.

Step 2: We use two aliases: "e" for the employee and "m" for the manager. The join condition is: e.manager_id = m.employee_id.

Step 3: The filter is simple: the employee salary must be more than the manager salary. e.salary > m.salary.

Step 4: We show first name, last name, and salary for both the employee and the manager.

SQL Query:

```
SELECT e.first_name || ' ' || e.last_name AS employee_name,
       e.salary AS employee_salary,
       m.first_name || ' ' || m.last_name AS manager_name,
       m.salary AS manager_salary
FROM employees e
INNER JOIN employees m
      ON e.manager_id = m.employee_id
WHERE e.salary > m.salary
ORDER BY e.salary DESC;
```

Example 3: Department with Most Employees

Task:

Find the department name that has the most employees. Show the department name and the number of employees.

Step-by-Step:

Step 1: We need to count employees in each department. So we use COUNT(*) with GROUP BY department_id.

Step 2: We join EMPLOYEES with DEPARTMENTS to get the department name, not just the ID.

Step 3: We order by count DESC and take only the first row with FETCH FIRST 1 ROW ONLY (or ROWNUM = 1 in older Oracle).

SQL Query:

```
SELECT d.department_name,
       COUNT(e.employee_id) AS total_employees
FROM employees e
INNER JOIN departments d
      ON e.department_id = d.department_id
ORDER BY total_employees DESC
FETCH FIRST 1 ROW ONLY;
```

```
GROUP BY d.department_name
ORDER BY total_employees DESC
FETCH FIRST 1 ROW ONLY;
```

Example 4: Salary Gap - Highest and Lowest by Job

Task:

For each job title, find the difference between the highest salary and the lowest salary. Show only jobs where the gap is more than 5000.

Step-by-Step:

Step 1: We need the job title, so we join EMPLOYEES with JOBS on job_id.

Step 2: For each job, we find the max salary (MAX) and min salary (MIN). This needs GROUP BY job_title.

Step 3: The gap = MAX(salary) - MIN(salary). We can calculate this in SELECT.

Step 4: We filter with HAVING, because the filter is on the result of an aggregate function (MAX - MIN > 5000).

SQL Query:

```
SELECT j.job_title,
       MAX(e.salary) AS highest_salary,
       MIN(e.salary) AS lowest_salary,
       MAX(e.salary) - MIN(e.salary) AS salary_gap
FROM employees e
INNER JOIN jobs j
      ON e.job_id = j.job_id
GROUP BY j.job_title
HAVING MAX(e.salary) - MIN(e.salary) > 5000
ORDER BY salary_gap DESC;
```

Example 5: Employees Earning Above Average

Task:

Find all employees who earn more than the average salary of the whole company. Show employee name, job title, salary, and how much more they earn than the average.

Step-by-Step:

Step 1: First, we need the average salary of the whole company. We can get this with a subquery: SELECT AVG(salary) FROM employees.

Step 2: We use this subquery in the WHERE clause. Each employee's salary must be > the result of the subquery.

Step 3: We join EMPLOYEES with JOBS to show the job title. We also calculate the difference: salary - average.

Step 4: In the SELECT list, we can write the subquery again to show the difference: salary - (SELECT AVG(salary) FROM employees).

SQL Query:

```
SELECT e.first_name || ' ' || e.last_name AS employee_name,
       j.job_title,
       e.salary,
       e.salary - (SELECT AVG(salary) FROM employees) AS above_avg
FROM employees e
INNER JOIN jobs j
      ON e.job_id = j.job_id
WHERE e.salary > (SELECT AVG(salary) FROM employees)
ORDER BY above_avg DESC;
```

PART 2: Practice Tasks

Now it is your turn! Solve these 15 tasks. Write your SQL queries on paper or in a database tool. Use the Oracle HR schema. Good luck!

Task 1	Total Salary Cost	JOIN + GROUP BY + SUM
--------	-------------------	-----------------------

Find the total salary cost (sum of all salaries) for each department. Show department name and total salary. Order by total salary from high to low.

Task 2	Employee Count by Department	JOIN + GROUP BY + HAVING + COUNT
--------	------------------------------	----------------------------------

Count how many employees work in each department. Show department name and count. Show only departments with more than 5 employees.

Task 3	Highest Paid Employee in Each Department	JOIN + GROUP BY + MAX + Subquery
--------	--	----------------------------------

Find the employee with the highest salary in each department. Show department name, employee name, and salary.

Task 4	Average Salary by Job Title	JOIN + GROUP BY + AVG + ORDER BY
--------	-----------------------------	----------------------------------

Find the average salary for each job title. Show job title and average salary. Order by average salary from low to high.

Task 5	Employees Without a Department	LEFT JOIN + IS NULL
--------	--------------------------------	---------------------

Find all employees who do not have a department (department_id is null). Show their name and job title.

Task 6	Salary Range Analysis	JOIN + GROUP BY + MIN + MAX + AVG
--------	-----------------------	-----------------------------------

For each job title, show the minimum salary, maximum salary, and average salary. Use the JOBS table to show job titles. Order by the salary range (max - min) from big to small.

Task 7	New Hires in 2024-2025	WHERE + BETWEEN or date filter + JOIN
--------	------------------------	---------------------------------------

Find employees hired in the years 2024 and 2025. Show their name, hire date, and department name. Order by hire date (newest first).

Task 8	Departments with High Average Salary	GROUP BY + HAVING + Subquery
--------	--------------------------------------	------------------------------

Find departments where the average salary is higher than the company average. Show department name and average salary. Use a subquery for the company average.

Task 9	Employees Who Changed Jobs	Subquery + JOIN (employees, jobs, job_history)
--------	----------------------------	--

Find employees who have a record in the JOB_HISTORY table. This means they changed their job. Show employee name, old job, new job, and the dates.

Task 10	Salary Comparison: Employee vs Department Average	Subquery + JOIN + Arithmetic
---------	---	------------------------------

For each employee, show their salary and their department average salary. Show the difference. Show only employees who earn less than their department average.

Task 11	Managers and Their Team Size	SELF JOIN + GROUP BY + HAVING + COUNT
---------	------------------------------	---------------------------------------

For each manager, count how many employees they manage. Show manager name and team size. Show only managers with more than 3 employees.

Task 12	Salary Percentile: Top 5 Earners	ORDER BY + FETCH FIRST (or ROWNUM)
---------	----------------------------------	------------------------------------

Find the top 5 highest paid employees in the whole company. Show their name, job title, and salary.

Task 13	Salary Distribution by Department	CASE WHEN + GROUP BY + SUM
---------	-----------------------------------	----------------------------

For each department, count how many employees earn more than 10000 and how many earn 10000 or less. Show department name and both counts.

Task 14**Average Years of Service by Department***Date Functions + GROUP BY + AVG + JOIN*

Find the average number of years employees have worked in each department. Use the HIRE_DATE column. Show department name and average years. Order by longest average service first.

Task 15**Cross-Department Salary Analysis***Subquery + Self JOIN + HAVING*

Find all pairs of departments where the average salary difference is more than 3000. Use a self-join on a subquery that calculates average salary per department.